



Feature Engineering : Transformer les données brutes en signal utile

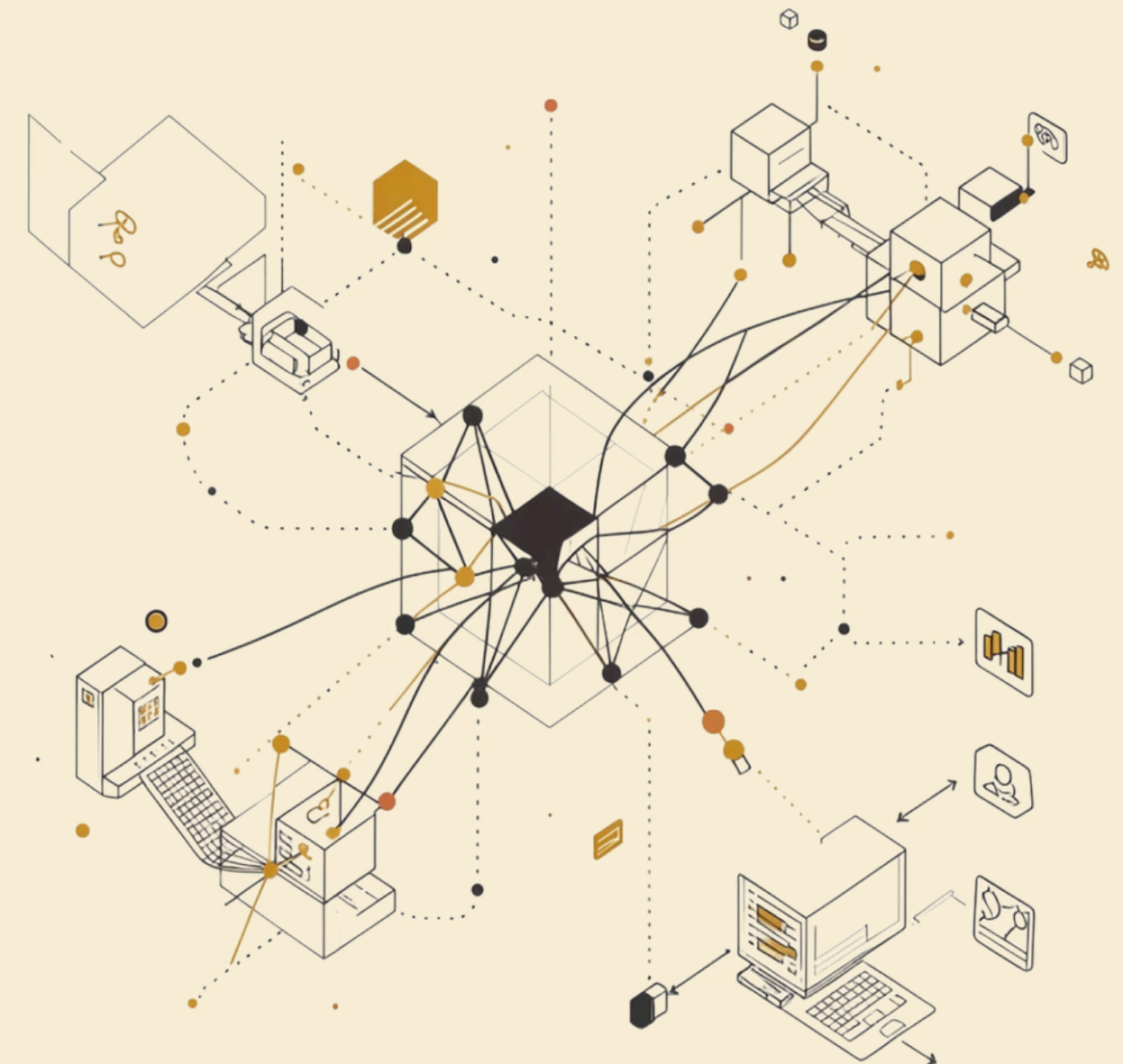
Découvrez comment passer de l'information brute à des indicateurs pertinents pour des modèles de machine learning plus performants.

Présenté par Laila EL JANI

Une colonne brute n'est pas toujours une information utile

Un modèle de machine learning reçoit des nombres et des catégories. Il ne comprend pas automatiquement le contexte, le comportement habituel, la temporalité, ni la différence entre ce qui est normal et ce qui est suspect. **Une donnée brute décrit ce qui s'est passé. Une feature bien construite explique pourquoi c'est important.**

i À retenir : Quand vous voyez une colonne comme amount, demandez-vous : élevé par rapport à quoi ? Par rapport à qui ? À quel moment ? Dans quel contexte ? C'est exactement là que commence le **feature engineering**.



Le montant seul ne suffit pas

user_id	amount	country	hour	is_fraud
U001	120	MA	14	0
U001	135	MA	16	0
U001	980	ES	3	1
U002	950	MA	11	0
U002	1000	MA	12	0

Données brutes disponibles

Variable	Signification
user_id	Identifiant du client
amount	Montant de la transaction
country	Pays de la transaction
hour	Heure de la transaction
is_fraud	Transaction frauduleuse ou non

Le problème du montant brut

Un montant de **1 000 DH** peut être :

- Normal pour un client professionnel
- Suspect pour un étudiant
- Banal pour un client VIP
- Très étrange si le client dépense habituellement 100 DH



Message clé : Le montant n'a pas de sens sans référence comportementale.

Montant comparé au comportement habituel

La feature créée

$$\text{amount_vs_user_avg} = \text{amount} / \text{avg_amount_user}$$

Cette feature exprime le montant de la transaction comme un **ratio par rapport à la moyenne historique** du client.

Exemple concret

user_id	amount	avg_amount_user	amount_vs_user_avg
U001	100	120	0.83
U002	900	100	9.00
U003	850	800	1.06

U002 déclenche un signal fort : son montant est **9 fois supérieur** à son habitude, même si 900 semble "normal" en valeur absolue.

amount

Montant brut — le modèle voit un nombre sans contexte

amount_vs_user_avg

Niveau d'anomalie comportementale — le modèle comprend l'écart à la norme

Pays inhabituel & transaction de nuit

🌐 Feature : changement géographique

```
is_unusual_country = country != usual_country
```

user_id	country / usual	is_unusual
U001	MA / MA	0
U002	ES / MA	1
U003	FR / FR	0

Le pays seul n'est pas suspect — c'est la **rupture par rapport aux habitudes** qui l'est.

🌙 Feature : transaction nocturne

```
is_night_transaction = hour < 6
```

user_id	hour	is_night
U001	14h	0
U002	2h	1
U003	23h	0

Une action à 2h du matin n'a pas la même signification qu'à 14h. On transforme une heure brute en **indicateur comportemental**.

Détecter un rythme anormal d'activité

Une fraude est rarement une transaction isolée. C'est souvent une **séquence rapide d'actions** destinée à maximiser les dégâts avant détection. Le feature engineering permet au modèle de capturer cette dynamique.

Donnée brute

tx_last_hour — Nombre de transactions pendant la dernière heure. Le modèle voit un nombre, mais ne comprend pas si c'est anormal.

Feature créée

high_velocity = tx_last_hour > 5 — Le modèle reçoit un signal binaire clair : activité normale (0) ou rythme suspect (1).

Exemple d'application

U001 : 1 tx → 0 
| U002 : 8 tx → 1  | U003 : 2 tx → 0 .
La vitesse révèle une tentative de fraude en série.

Avant / Après Feature Engineering

Le modèle ne voit plus seulement une transaction. Il voit un **comportement**. Chaque feature ajoute une dimension sémantique que la donnée brute ne portait pas.

Donnée brute	Feature créée	Information ajoutée
amount	amount_vs_user_avg	Anomalie par rapport au profil client
country	is_unusual_country	Rupture géographique comportementale
hour	is_night_transaction	Contexte temporel sensible
tx_last_hour	high_velocity	Activité anormalement rapide

4

Features créées

À partir de données brutes initiales

4

Dimensions ajoutées

Contexte, anomalie, temporalité, vitesse

1

Objectif

Transformer des nombres en signaux métier

Un appel au support n'a pas toujours le même sens

client_id	support_calls	tenure_months	churn
C001	5	2	1
C002	5	48	0
C003	1	1	1
C004	3	36	0

Données brutes disponibles

Variable	Signification
client_id	Identifiant client
support_calls	Nombre total d'appels au support
tenure_months	Ancienneté du client (en mois)
churn	Client parti ou non

Le problème du volume brut

support_calls = 5 peut signifier :

- 5 appels en 2 mois → **insatisfaction aiguë**, churn très probable
- 5 appels en 5 ans → **comportement normal**, client fidèle

⚠ Le chiffre brut dit : "ce client a appelé 5 fois." La feature dira : "ce client appelle très souvent *par rapport à son ancienneté*."

Appels support par mois : de la quantité à l'intensité

La feature créée

```
support_calls_per_month = support_calls /  
tenure_months
```

Cette feature normalise le volume d'appels par la durée de vie du client, produisant un **indicateur d'intensité d'insatisfaction**.

Exemple concret

client_id	support_calls	tenure_months	calls/month	churn
C001	5	2	2.50	1 
C002	5	48	0.10	0
C003	3	36	0.08	0

C001 et C002 ont le même nombre d'appels, mais des profils radicalement différents. La feature révèle ce que le brut cache.

support_calls Volume brut — sans contexte temporel	support_calls_per_month Intensité de l'insatisfaction — signal prédictif fort
--	---

Visiter beaucoup de pages ne veut pas toujours dire acheter

session_id	pages_visited	time_spent_seconds	purchase
S001	20	60	0
S002	20	900	1
S003	3	30	0
S004	10	600	1

Données brutes disponibles

Variable	Signification
pages_visited	Nombre de pages consultées
time_spent_seconds	Durée de la session
purchase	Achat réalisé ou non

Le problème

Deux utilisateurs visitent **20 pages** :

- L'un en **60 secondes** → comportement robotique, probablement un bot
- L'autre en **15 minutes** → navigation humaine, intention d'achat réelle



Feature à créer : $\text{pages_per_minute} = \text{pages_visited} / (\text{time_spent_seconds} / 60)$ — un ratio qui distingue la navigation humaine d'un comportement automatisé.

Pages par minute : du volume au comportement

Le nombre de pages visitées seul peut être trompeur. Pour détecter l'intention réelle ou un comportement anormal, il faut mesurer le rythme de navigation.

La feature créée

```
pages_per_minute = pages_visited / (time_spent_seconds / 60)
```

Ce ratio transforme l'activité brute en un indicateur d'intensité de navigation, révélant si l'utilisateur est attentif, hésitant, ou s'il s'agit d'un comportement automatisé.

Exemple concret

session_id	pages_visited	time_spent_seconds	pages_per_minute	purchase
S001	20	60	20.00	0
S002	20	900	1.33	1

Les sessions S001 et S002 ont le même nombre de pages visitées, mais la feature pages_per_minute révèle une intention radicalement différente.

Variable brute

pages_visited — ne donne qu'une **activité globale**.

Nouvelle lecture

pages_per_minute — révèle un **comportement rapide, hésitant ou anormal**.

Cas logistique : l’heure brute trompe le modèle

L'heure d'un événement, comme une livraison, est une information cruciale. Cependant, utilisée telle quelle, la variable hour peut induire un modèle en erreur en masquant des proximités temporelles importantes.

Données brutes

delivery_id	hour	delay_minutes
D001	22	30
D002	23	35
D003	0	40
D004	1	38
D005	12	5

Le problème de la discontinuité numérique

Pour un modèle, **23** et **0** sont numériquement **très éloignés**.

Dans la réalité logistique, **23h** et **0h** sont des moments **temporellement proches** avec des comportements similaires.

Nouvelle feature : encodage cyclique de l'heure

Les features créées

Pour que le modèle comprenne la nature cyclique du temps, nous transformons l'heure en coordonnées sur un cercle en utilisant les fonctions sinus et cosinus.

$$hour_sin = \sin(2\pi \times hour/24)$$

$$hour_cos = \cos(2\pi \times hour/24)$$

Cela permet de représenter les heures de manière continue où 23h et 0h sont perçues comme proches.

Exemple concret

hour	hour_sin	hour_cos
23	-0.26	0.97
0	0.00	1.00
1	0.26	0.97
12	0.00	-1.00

Les valeurs sinusoïdales et cosinusoidales de 23h et 0h sont très proches, résolvant le problème de discontinuité numérique.

Avant

Heure comme nombre linéaire

Pour un modèle, 23 et 0 sont numériquement **très éloignés**, ignorant la proximité temporelle.

Après

Heure comme cycle

Grâce à l'encodage cyclique, 23 et 0 deviennent **proches**, reflétant la réalité temporelle et les comportements associés.

"Le modèle ne sait pas que le temps tourne en boucle. L'encodage sinus-cosinus permet de représenter correctement cette cyclicité."



EXEMPLE 5 — MAINTENANCE INDUSTRIELLE

Cas maintenance : une panne est souvent une évolution

En maintenance prédictive, la détection d'une panne ne repose pas toujours sur un seuil unique. Souvent, c'est l'évolution des signaux qui préfigure le problème, bien avant que la "valeur brute" ne devienne critique.

Données brutes : l'instant T ne dit pas tout

timestamp	température (°C)	vibration (G)	panne
10:00	70	0.20	0
10:05	72	0.25	0
10:10	75	0.35	0
10:15	81	0.60	1

La panne survient à 10:15. Mais un simple seuil sur la température ou la vibration aurait pu manquer les signaux d'alerte progressifs.

Le problème : la valeur actuelle est insuffisante

Se concentrer uniquement sur la valeur instantanée des capteurs peut masquer les indicateurs clés d'une défaillance imminente. La magie du Feature Engineering réside dans la capacité à transformer ces séquences en signaux prédictifs.

Ce qui compte souvent

- La **variation** des mesures
- L'**accélération** des changements
- La **tendance** évolutive
- L'**instabilité** ou la volatilité

Nouvelles features : variation et tendance

Pour que le modèle anticipe une panne, il doit comprendre non seulement l'état actuel mais aussi la dynamique des capteurs. Nous créons des features qui mesurent la variation et la tendance.

Les features créées

```
temperature_delta = temperature - temperature_previous
vibration_rolling_mean = moyenne des vibrations récentes
```

Ces features transforment des mesures isolées en indicateurs de **trajectoire** et d'instabilité, permettant de détecter une dérive avant qu'elle ne devienne critique.

Exemple concret

timesta mp	temp	temp_ delta	vib	vib_mean	panne
10:00	70	—	0.20	0.20	0
10:05	72	2	0.25	0.22	0
10:10	75	3	0.35	0.27	0
10:15	81	6	0.60	0.40	1

La temp_delta et la vib_mean montrent une nette accélération avant la panne, là où les valeurs brutes étaient encore acceptables à 10:10.

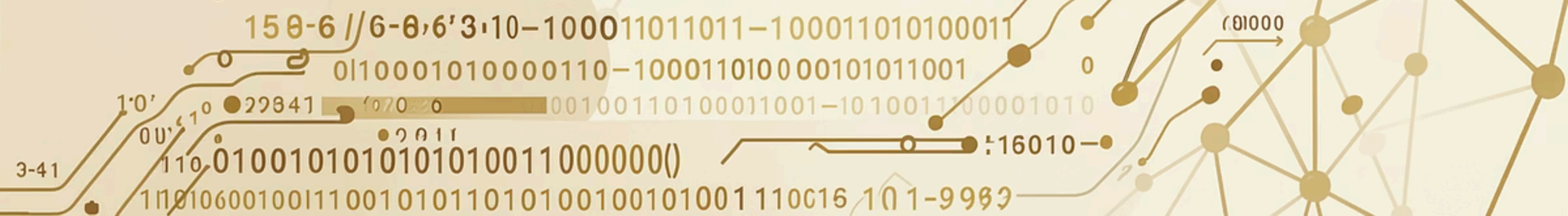
Variable brute

Température / Vibration instantanée
Indique un état ponctuel, potentiellement trop tardif pour la maintenance prédictive.

Nouvelles features

Variation (delta) / Tendance (moyenne glissante)
Révèle la vitesse de dégradation et l'instabilité, permettant une intervention proactive.

"Une machine ne tombe pas en panne parce qu'une température existe. Elle tombe en panne parce qu'un système dérive. Le feature engineering transforme des mesures isolées en trajectoire."



EXEMPLE 6 — TEXTE CLIENT

Cas texte : un commentaire doit devenir un signal numérique

Les données textuelles brutes, telles que les commentaires clients, sont riches en informations. Cependant, les modèles de machine learning ne peuvent pas les interpréter directement. Il est indispensable de les transformer en un format numérique exploitable.

Données brutes

client_id	comment	churn
C001	je veux résilier mon abonnement	1
C002	service excellent merci	0
C003	trop cher je vais partir	1
C004	bonne qualité réseau	0

Le problème : les mots ne sont pas des nombres

Un modèle d'apprentissage automatique opère sur des chiffres. Une phrase comme "je veux résilier mon abonnement" n'a aucun sens pour lui telle quelle. Sans un **Feature Engineering** adapté, cette information cruciale reste inexploitée.



FEATURE ENGINEERING TEXTE

Nouvelle feature : présence de mots à risque

Les commentaires clients contiennent des indices précieux sur leur satisfaction ou leur intention. Pour les rendre exploitables par un modèle, nous identifions des mots-clés et les transformons en signaux numériques binaires.

Texte brut

- Phrase libre et non structurée
- Sens implicite, difficile à quantifier

Feature créée

- Signal numérique exploitable (0 ou 1)
- Indicateur direct de risque ou de satisfaction

Features créées

Chaque mot ou expression clé identifié comme "à risque" ou positif devient une nouvelle colonne (feature) binaire (0 ou 1) :

- mot_resilier : présent si le client parle de résiliation.
- mot_cher : présent si le client juge le prix élevé.
- mot_excellent : présent si le client exprime une grande satisfaction.

Cela permet au modèle de directement capter l'intention derrière le texte.

Exemple concret

je veux résilier mon abonnement	1	0	0	1	
service excellent merci	0	0	1	0	trop
cher je vais partir	0	1	0	1	bonne
qualité réseau	0	0	0	0	

Les mots-clés sont transformés en indicateurs clairs, directement utilisables par les algorithmes de ML.

"Le texte contient souvent des signaux très puissants. Mais tant qu'on ne les transforme pas, ils restent invisibles pour le modèle."

Cas merchant_id : trop de catégories tuent le signal

Le merchant_id (identifiant du commerçant) est une information vitale pour détecter la fraude. Chaque transaction est associée à un commerçant, et des anomalies spécifiques à certains commerçants peuvent être des indicateurs clés. Cependant, la nature même de cette donnée présente un défi majeur pour le Feature Engineering traditionnel.

Données brutes

transaction_id	merchant_id	is_fraud
T001	M001	0
T002	M001	0
T003	M002	1
T004	M002	1
T005	M003	0

Le problème : une cardinalité trop élevée

Le merchant_id peut facilement avoir **10 000 valeurs différentes**, voire beaucoup plus dans le cas de plateformes de commerce électronique mondiales. Une approche classique comme le one-hot encoding transformerait chaque identifiant unique en une colonne binaire séparée (0 ou 1). Cela créerait:

- Un nombre **énorme de colonnes** (10 000 ou plus).
- Un jeu de données **extrêmement clairsemé** (sparse), la plupart des valeurs étant nulles.
- Des problèmes de **performance** et de **surapprentissage** pour les modèles.

Sans un traitement spécifique, cette information essentielle reste difficilement utilisable.

Nouvelle feature : score de risque marchand

Plutôt que de traiter chaque merchant_id comme une catégorie distincte, nous pouvons agréger les informations pour créer un indicateur synthétique de risque.

Feature créée

merchant_risk_score = taux historique de fraude du marchand

Ce score représente la proportion de transactions frauduleuses associées à un commerçant donné, offrant une mesure numérique de son profil de risque.

Exemple

merchant_id	fraud_count	total_tx	merchant_risk_score
M001	0	2	0.00
M002	2	2	1.00
M003	0	1	0.00

Avant

- Identifiant sans signification directe
- Beaucoup de colonnes possibles (si one-hot encoded)

Après

- Niveau de risque historique
- Une variable numérique synthétique

"On ne veut pas que le modèle mémorise le nom du marchand. On veut qu'il comprenne le niveau de risque associé à ce marchand."

"Et attention : si ce score est calculé sur train + test, vous avez créé du **leakage**."

Ce que le Feature Engineering ajoute réellement

Le Feature Engineering ne se contente pas de nettoyer les données brutes ; il les enrichit en transformant des informations descriptives en signaux prédictifs puissants pour les modèles.

Donnée brute	Feature Engineering	Nouvelle information
amount	amount_vs_user_avg	Anomalie individuelle
country	is_unusual_country	Rupture géographique
hour	hour_sin, hour_cos	Cyclicité temporelle
support_calls	support_calls_per_month	Intensité d’insatisfaction
pages_visited	pages_per_minute	Rythme de navigation
temperature	temperature_delta	Variation brutale
vibration	rolling_mean	Instabilité récente
comment	mots clés / TF-IDF	Signal sémantique
merchant_id	merchant_risk_score	Risque historique

"Le Feature Engineering transforme des colonnes descriptives en signaux décisionnels."

À vous : transformez les données en signal

Il est maintenant temps de mettre en pratique les concepts de Feature Engineering. Observez attentivement les données brutes fournies et identifiez les informations qui, une fois transformées, pourraient révéler un comportement suspect.

Données brutes

client_id	amount	hour	country	avg_amount_user	usual_country	tx_last_hour
U001	100	14	MA	120	MA	1
U002	900	2	ES	100	MA	8
U003	850	11	MA	800	MA	1
U004	70	23	FR	60	FR	2

Consigne

Créez 4 nouvelles features

Elles doivent rendre le comportement suspect de certaines transactions plus visible pour un modèle.

Correction : les features qui révèlent l'anomalie

En appliquant les principes de Feature Engineering, nous transformons des données brutes souvent anodines en signaux clairs et exploitables, permettant aux modèles de détecter les comportements anormaux avec une plus grande précision.

Features créées

1

amount_vs_avg_user = amount / avg_amount_user

2

is_night_transaction = hour < 6

3

is_unusual_country = country != usual_country

4

high_velocity = tx_last_hour > 5

Tableau transformé

client_id	amount_vs_avg_user	is_night_transaction	is_unusual_country	high_velocity
U001	0.83	0	0	0
U002	9.00	1	1	1
U003	1.06	0	0	0
U004	1.17	0	0	0

Dans le tableau brut, la transaction U002 est simplement un montant élevé. Dans le tableau transformé, elle devient clairement un **comportement suspect** grâce à l'enrichissement des features.

Ce que le Feature Engineering apporte vraiment

À travers trois cas concrets — fraude bancaire, churn télécom, e-commerce — un principe commun émerge : **une feature bien construite transforme une donnée brute en signal métier interprétable**. Le modèle apprend mieux, plus vite, et avec moins de données.



Ajouter du contexte

Un montant, un pays, une heure n'ont de sens que par rapport à un comportement de référence. Le feature engineering crée cette référence.



Coder l'anomalie

Les features les plus puissantes ne décrivent pas l'état — elles codent **l'écart à la normale**. C'est cet écart qui est prédictif.



Normaliser par la durée

Diviser un volume par une durée (appels/mois, pages/minute) transforme une quantité brute en intensité comportementale.



Encoder la logique métier

Le feature engineering est l'art de traduire votre compréhension du problème en variables que le modèle peut exploiter directement.

"Le feature engineering, c'est l'art de transformer des données brutes en histoires que votre modèle peut comprendre."

CONCLUSION FORTE

Le modèle ne découvre pas tout seul le sens métier

Un modèle ne comprend pas automatiquement:

- Ce qui est rare
- Ce qui est anormal
- Ce qui est récent
- Ce qui est cyclique
- Ce qui est contextuel
- Ce qui est dangereux

Le modèle ne voit que des colonnes. **Le Feature Engineering transforme ces colonnes en intelligence exploitable.**